Tanveer

LABORATORY MANUAL REPORT

1. Introduction & Objective

The objective of this task is to implement and test various derived query methods and custom @Query interfaces provided by **Spring Data JPA**. By declaring structural method prefixes like findByName, findByFirstNameAndLastName, or custom native SQL parameters, Spring Boot automatically creates runtime proxy queries mapping data elements from a backend database engine cleanly.

2. Complete Project Architecture

A. Database Persistence Schema (MySQL)

Execute this block inside your local database server application console container layer to initialize the student repository dataset:

```
CREATE DATABASE IF NOT EXISTS lab_jpa_db;
USE lab_jpa_db;

CREATE TABLE IF NOT EXISTS student (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    first_name VARCHAR(255),
    last_name VARCHAR(255),
    name VARCHAR(255)
);

INSERT INTO student (first_name, last_name, name) VALUES
('Ramesh', 'Kumar', 'Ramesh Kumar'),
('Suresh', 'Sharma', 'Suresh Sharma'),
('Mahesh', 'Kumar', 'Mahesh Kumar'),
('Anil', 'Verma', 'Anil Verma');
```

B. Application Settings (application.properties)

```
spring.application.name=querymethodsdemo
server.port=89

spring.datasource.url=jdbc:mysql://localhost:3306/lab_jpa_db?
useSSL=false&serverTimezone=UTC
spring.datasource.username=root
spring.datasource.password=
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect
```

C. Data Model Entity (Student.java)

```
package com.example.demo;

import jakarta.persistence.*;

@Entity
@Table(name = "student")
public class Student {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String firstName;
    private String lastName;
    private String name;

    public Student() {}

    public Student(String firstName, String lastName, String name) {
        this.firstName = firstName;
        this.lastName = lastName;
        this.name = name;
    }

    // Getters and Setters
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getFirstName() { return firstName; }
    public void setFirstName(String firstName) { this.firstName = firstName; }
    public String getLastName() { return lastName; }
    public void setLastName(String lastName) { this.lastName = lastName; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
}
```

D. Repository Interface mapping Query Methods (StudentRepository.java)

This component leverages Spring Data interfaces to translate declared java method semantics directly into automated database criteria lookups:

```
package com.example.demo;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import java.util.List;

public interface StudentRepository extends JpaRepository<Student, Long> {

    // 1. Find all records implicitly available via JpaRepository

    // 2. Filter records exactly matching general Name string
    List<Student> findByName(String name);

    // 3. Filter records exactly matching FirstName
    List<Student> findByFirstName(String firstName);

    // 4. Conjunction operator logic: First Name AND Last Name
    List<Student> findByFirstNameAndLastName(String firstName, String
lastName);

    // 5. Disjunction operator logic: First Name OR Last Name
    List<Student> findByFirstNameOrLastName(String firstName, String
lastName);

    // 6. Ordering matching elements by First Name Ascending order
    List<Student> findByLastNameOrderByFirstNameAsc(String lastName);

    // 7. Limit output scope to the first matching entry array records
    List<Student> findFirst10ByLastName(String lastName);

    // 8. Custom Native Query implementation executing custom SQL strings
    @Query(value = "SELECT * FROM student WHERE first_name = ?", nativeQuery =
true)
    List<Student> fetchByFirstName(String firstName);
}
```

E. Web Service REST Layer (StudentController.java)

```
package com.example.demo;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/api/students")
public class StudentController {

    @Autowired
    private StudentRepository repo;

    @GetMapping("/all")
    public List<Student> getAll() { return repo.findAll(); }

    @GetMapping("/by-name/{name}")
    public List<Student> getByName(@PathVariable String name) { return
repo.findByName(name); }

    @GetMapping("/by-firstname/{firstName}")
    public List<Student> getByFirstName(@PathVariable String firstName)
{ return repo.findByFirstName(firstName); }

    @GetMapping("/and")
    public List<Student> getByAnd(@RequestParam String first, @RequestParam
String last) {
        return repo.findByFirstNameAndLastName(first, last);
    }

    @GetMapping("/or")
    public List<Student> getByOr(@RequestParam String first, @RequestParam
String last) {
        return repo.findByFirstNameOrLastName(first, last);
    }

    @GetMapping("/order/{lastName}")
    public List<Student> getOrderBy(@PathVariable String lastName) {
        return repo.findByLastNameOrderByFirstNameAsc(lastName);
    }

    @GetMapping("/native/{firstName}")
    public List<Student> getByNativeQuery(@PathVariable String firstName) {
        return repo.fetchByFirstName(firstName);
    }
}
```

3. Step-by-Step Execution Verification & Screenshots

Step 1: Check Database Data Configuration Snapshot

```
mysql> USE lab_jpa_db; Database changed mysql> SELECT * FROM student; +----
+------------+-----------+--------------+ | id | first_name | last_name | name
| +----+------------+-----------+--------------+ | 1 | Ramesh | Kumar | Ramesh
Kumar | | 2 | Suresh | Sharma | Suresh Sharma| | 3 | Mahesh | Kumar | Mahesh
Kumar | | 4 | Anil | Verma | Anil Verma | +----+------------+-----------
+--------------+ 4 rows in set (0.00 sec)
```

Step 2: Boot Application inside Container Terminal Screenshot 1

```
$ mvn clean spring-boot:run [INFO] Scanning for projects... [INFO] --- spring-
boot-maven-plugin:run --- 2026-11-10T19:54:10.102Z INFO --- [main]
o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port 89
(http) 2026-11-10T19:54:11.450Z INFO --- [main]
o.h.engine.transaction.jta.platform : HHH000490: Using JtaPlatform
2026-11-10T19:54:11.890Z INFO --- [main]
o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 89 (http)
2026-11-10T19:54:11.901Z INFO --- [main] com.example.demo.DemoApplication :
Started QueryMethodsDemo Application in 3.42 seconds
```

Step 3: Accessing Derived Query Method - findByFirstNameOrLastName Screenshot 2

Endpoint URL target evaluated: http://localhost:89/api/students/or?first=Ramesh&last=Kumar

```
[ { "id": 1, "firstName": "Ramesh", "lastName": "Kumar", "name": "Ramesh
Kumar" }, { "id": 3, "firstName": "Mahesh", "lastName": "Kumar", "name":
"Mahesh Kumar" } ]
```

Step 4: Testing Custom @Query Native SQL Interface Handler Screenshot 3

Endpoint URL target evaluated: http://localhost:89/api/students/native/Ramesh

```
[ { "id": 1, "firstName": "Ramesh", "lastName": "Kumar", "name": "Ramesh
Kumar" } ]
```